SMU
SINGAPORE MANAGEMENT UNIVERSITY

AY 2024-25 Term 1 Examination

COR1704 Computational Thinking and Programming

INSTRUCTIONS

Please read this page only. Do not open the paper until instructed.

1. This is a **closed-book** examination. Before you begin, please remove everything from your workspace except for pens, pencils, and erasers. No use of calculators is allowed.

2. The time allowed for this examination paper is **TWO** hours.

3. This paper comprises 33 pages, including the cover page and 3 blank pages at the end.

4. This examination consists of 2 sections.
   a. **Section A** consists of 10 multiple-choice questions (MCQ). Each MCQ is worth 2 marks. For each question, select the **BEST** choice. (Each MCQ has only a single correct answer.)
   b. **Section B** consists of 8 short answer questions.

5. You are required to enter all answers (both sections **A** and **B**) in the eLearn **COR1704-Computational Thinking and Programming-Merged Section**.

6. You must return all parts of this examination paper to the invigilators. Missing examination papers or parts thereof will be considered cheating.

	Marks	Awarded
Section A (MCQs):	20	
Question 11	8	
Question 12	7	
Question 13	7	
Question 14	3	
Question 15	6	
Question 16	7	
Question 17	4	
Question 18	8	
TOTAL	70	

THIS PAGE IS INTENTIONALLY LEFT BLANK.

Section A [MCQs]

1. **[Difficulty: *]** What is the output of the following program?

```
output = 0
for i in range(2, 6):
    output = i*2
    print(output, end = '')
```

[ ] A. 2468

[ ] B. 4681012

[ ] C. 2345

[x] D. 46810

[ ] E. None of the given options are correct.

2. **[Difficulty: *]** Which of the following code snippets generates exactly the following output?

```
IS1704
IS1704
```

[ ] A.
```
for i in range(1, 3):
    print("IS1704\n")
```

[ ] B.
```
counter = 1
while (counter > 0):
    print("IS1704")
    counter = counter + 1
```

[ ] C.
```
my_str = 'IS1704'
while len(my_str) == 2:
    print(my_str)
```

[x] D.
```
str_list = ["IS1704", "IS1704", "IS1704"]
i = 0
while i < 2:
    print(str_list[i])
    i = i + 1
```

[ ] E.
```
a_str = "IS1704"
for i in range(2, 3):
    print(a_str)
```

3. **[Difficulty: *]** What is the output of the following program?

```
my_str = '12ab45'
num = 10

x = float(my_str[1:2])
print(x)
print(int(x) + num)
```

[ ] A.
```
2.0
22
```

[x] B.
```
2.0
12
```

[ ] C. Execution error only

[ ] D.
```
2
```
Execution error

[ ] E.
```
2
12
```

4. **[Difficulty: *]** The `while` loop in the below code must be terminated when the value of `x` is less than `8`, and the value of `y` is more than `2`.

Which code snippet should be inserted into the blank?

```
x = int(input('Enter x:'))
y = int(input('Enter y:'))

while _____________________:
    print("Please enter again!")
    x = int(input('Enter x again:'))
    y = int(input('Enter y again:'))
```

[ ] A. `not (x > 8 and y < 2)`

[ ] B. `x < 8 or y > 2`

[ ] C. `not (x <=8 and y >=2)`

[x] D. `x >= 8 or y <= 2`

[ ] E. None of the given options are correct.

5. **[Difficulty: *]** What is the output of the following program?

```
a_str = 'abcdefg'
if len(a_str[2:5] + a_str[::]) >= 11:
    print('Yes')
elif len(a_str[:2]) + len(a_str[2::2]) <= 5:
    print('No')
else:
    print('Done')
```

[ ] A. `Yes`

[ ] B. `Yes`
`Done`

[x] C. `No`

[ ] D. `No`
`Done`

[ ] E. `Done`

6. **[Difficulty: **]** What is the output of the following program?

```
my_str = 'qwertykeyboard'
list1 = [ len(my_str[::3]), len(my_str[:5]), my_str[9:] ]
list2 = list1
list3 = list2 + [1, 2, 3]
print(list1.append([11, 12, 13]))
print(list1)
print(list2)
print(list3[:-1])
```

[ ] A.
```
[5, 5, 'board', [11, 12, 13]]
[5, 5, 'board', [11, 12, 13]]
[5, 5, 'board']
[5, 5, 'board', 1, 2, 3]
```

[ ] B.
```
None
[5, 5, 'board', [11, 12, 13]]
[5, 5, 'board']
[5, 5, 'board', 1, 2]
```

[ ] C.
```
None
[5, 5, 'board', [11, 12, 13]]
[5, 5, 'board', [11, 12, 13]]
[3, 2, 1, 'board', 5, 5]
```

[x] D.
```
None
[5, 5, 'board', [11, 12, 13]]
[5, 5, 'board', [11, 12, 13]]
[5, 5, 'board', 1, 2]
```

[ ] E. None of the given options are correct.

7. **[Difficulty: **]** Consider the computational complexity of function implementations I, II, and III, and choose the correct option:

I.
```
def search_1(lst, target):
    for i in lst:
        if i == target:
            return True
    return False
```

II.
```
def search_2(lst, target):
    found = False
    for i in lst:
        for j in lst:
            if i == target:
                found = True
    return found
```

III.
```
def search_3(lst, target):
    for i in lst:
        for j in lst:
            for k in lst:
                if i == target:
                    return True
    return False
```

[ ] A. Implementation I has a worst-case complexity of $O(1)$.

[ ] B. Implementations I and III have the same worst-case complexity.

[x] C. Implementation II has a worst-case complexity of $O(n^2)$, where n is the length of `lst`.

[ ] D. All 3 implementations have the same worst-case complexity.

[ ] E. None of the given options are correct.

8. **[Difficulty: **]** What is the output of the following code?

```
def my_func(my_list):
    my_dict = {}
    for i in range(len(my_list)):
        new_list = my_list + [i]
        my_dict[i] = new_list[i::i+1]
    my_list.append(my_dict[0])
    return my_dict

a_list = [1, 2, 3, 4]
b_list = a_list
my_dict  = my_func(a_list)
print(my_dict)
print(a_list)
print(b_list)
```

[ ] A.
```
{0: [1, 2, 3, 4, 0], 1: [2, 4], 2: [3], 3: [4]}
[1, 2, 3, 4]
[1, 2, 3, 4]
```

[ ] B.
```
{0: [1, 2, 3, 4], 1: [2, 4], 2: [3], 3: []}
[1, 2, 3, 4, [1, 2, 3, 4]]
[1, 2, 3, 4, [1, 2, 3, 4]]
```

[ ] C.
```
{0: [1, 2, 3, 4, 0], 1: [2, 4], 2: [3], 3: [4]}
[1, 2, 3, 4, [1, 2, 3, 4, 0]]
[1, 2, 3, 4]
```

[x] D.
```
{0: [1, 2, 3, 4, 0], 1: [2, 4], 2: [3], 3: [4]}
[1, 2, 3, 4, [1, 2, 3, 4, 0]]
[1, 2, 3, 4, [1, 2, 3, 4, 0]]
```

[ ] E. None of the given options are correct.

9. **[Difficulty: **/***]** What is the output of the following code?

```
def create_dict(my_input):
    new_dict = {}
    key = my_input[0:]
    count = 6
    while(count > 0):
        if key not in new_dict:
            new_dict[key] = count
        else:
            new_dict[key] += count
        count -= 1
        if count > len(my_input) // 2:
            key = my_input[count - 1:]
        elif count > len(my_input) % 3:
            key = my_input[count + 1:]
        else:
            key = my_input[count:]
    return new_dict

my_dict = create_dict((1, 2, 3, 4, 5))
print(my_dict)

my_dict = create_dict((6, 7, 8))
print(my_dict)
```

[ ] A. `{(1, 2, 3, 4, 5): 6, (4, 5): 4, (3, 4, 5): 5, (2, 3, 4, 5): 1}`
`{(6, 7, 8): 6, (8,): 4, (7, 8): 2}`

[ ] B. `{(1, 2, 3, 4, 5): 6, (3, 4, 5): 5, (2, 3, 4, 5): 1}`
`{(6, 7, 8): 6, (): 9, (8,): 4, (7, 8): 2}`

[ ] C. `{(1, 2, 3, 4, 5): 6, (5,): 5, (4, 5): 4, (3, 4, 5): 5}`
`{(6, 7, 8): 6, (8,): 4, (7, 8): 2}`

[x] D. `{(1, 2, 3, 4, 5): 6, (5,): 5, (4, 5): 4, (3, 4, 5): 5, (2, 3, 4, 5): 1}`
`{(6, 7, 8): 6, (): 9, (8,): 4, (7, 8): 2}`

[ ] E. None of the given options are correct.

10. **[Difficulty: ***]** What is the output for the code below?

```
def process_lists(list1, list2, list3):
    result = []
    i = 0
    while i < max(len(list1), len(list2)):
        if i < len(list1) and i < len(list2):
            str1 = list1[i]
            str2 = list2[i]
            str3 = ''
            if i <= len(list3):
                str3 = list3[i]
            my_dict = {}
            j = 0
            while j < len(str1 + str2):
                if j < len(str1):
                    ch = str1[j].lower()
                else:
                    ch = str2[j - len(str1)].lower()
                # isalpha() returns True if ch is alphabetic, otherwise False
                if ch.isalpha():
                    if ch in my_dict:
                        my_dict[ch] += 1
                    else:
                        my_dict[ch] = 1
                j += 1
            k = 0
            while k < len(str3):
                ch = str3[k].lower()
                if ch.isalpha():
                    num = 0
                    for c in str3:
                        if c == ch.lower():
                            num += 1
                    if ch in my_dict:
                        if my_dict[ch] > num:
                            if ch not in result:
                                result.append(ch)
                k += 1
        i += 1
    return result
print(process_lists(["hello", "world", "python"], ["programming", "is",
"awesome"], ["hi", "there", "coder"]))
print(process_lists(["x1y2z3", "p4q5r6", "m7n8o9"], ["3z2y1x", "6r5q4p",
"9o8n7m"], ["yes", "agree", "mon", "more"]))
```

	A.	`['l', 'e']` `['x', 'r', 'm', 'o', 'n']`
x	B.	`['o', 'e']` `['y', 'r', 'm', 'o', 'n']`
	C.	`['h', 'e']` `['y', 'r', 'm', 'n', 'e']`
	D.	Execution error.
	E.	None of the given options are correct.

END of Section A

Section B

Question 11 [Difficulty: *] [8 marks]

You are a researcher studying ancient manuscripts from a forgotten civilization. The manuscripts contain encrypted messages that alternate between lowercase and uppercase letters. To help decode the messages, your task is to standardize them by alternating lowercase and uppercase letters based on their positions.

The task is to write a function `alternate_case(s)` that takes a string `s` and returns a new string where:

- Letters at even positions (starting from 0) are converted to lowercase.
- Letters at odd positions are converted to uppercase.
- Non-letter characters (such as spaces or punctuation marks) are left unchanged.

For example:

- `alternate_case('hello world!')` returns `'hElLo wOrLd!'`
- `alternate_case('python 101!')` returns `'pYtHoN 101!'`
- `alternate_case('') returns ''`

Type your answer in the space provided on eLearn.

```
# Solution
def alternate_case(s):
    return_str = ''

    for i in range(len(s)):
        if i % 2 == 0:
            return_str += s[i].lower()
        else:
            return_str += s[i].upper()

    return return_str
```

Question 12 [Difficulty: *] [7 marks]

Write a function called `get_products` that takes in two parameters:

- `start` (type: `int`): the initial number.
- `limit` (type: `int`): a positive integer.

This function should generate a list of products of integers in the form:

```
[start * 1, start * 2, start * 3, ...]
```

The function should stop when the length of the list is exactly `limit`. The function returns the list of products.

Examples:

- `get_products(1, 3)` returns `[1, 2, 3]`
- `get_products(1, 4)` returns `[1, 2, 3, 4]`
- `get_products(2, 4)` returns `[2, 4, 6, 8]`
- `get_products(-4, 6)` returns `[-4, -8, -12, -16, -20, -24]`

Type your answer in the space provided on eLearn.

```
# Solution
def get_products (start, limit):
    series = []

    for i in range(limit):
        product = start * (i + 1)
        series.append(product)

    return series
```

Question 13 [Difficulty: *] [7 marks]

FairPrice has asked you to help them with their software implementation. Write a function called `find_price` that takes in two parameters:

- `product_name` (type: `str`): The name of the product to search for.
- `inventory` (type: `list`): A list of tuples, where each tuple contains:
  - `product_name` (type: `str`): The name of the product. It is case-sensitive.
  - `price` (type: `float`): The regular price of the product.
  - `on_sale` (type: `bool`): Whether the product is on sale (`True` or `False`).

If the product is in the inventory and not on sale, the function should return the full price (type: `float`) of the product.
If the product is on sale, the function should apply a 15% discount.
The returned value should be rounded to two decimal places (hint: `round(3.1415, 2) => 3.14`).
If the product is not in the inventory, return `None`.

Examples:

Suppose the inventory is given as:

```
inventory = [
    ('apple', 0.99, False),
    ('banana', 0.49, True),
    ('milk', 1.99, False),
    ('dragonfruit', 5.99, True),
    ('unicorn tears', 299.99, True),
    ('space dust', 999.99, True)
]
```

- `find_price('banana', inventory)` should return `0.42` (15% discount on sale price).
- `find_price('milk', inventory)` should return `1.99` (not on sale).
- `find_price('unicorn tears', inventory)` should return `254.99` (15% discount on sale price).
- `find_price('caviar', inventory)` should return `None` (not in inventory).

Type your answer in the space provided on eLearn.

```
# Solution
def find_price(product_name, inventory):
    for item in inventory:
        name, price, on_sale = item
        if name == product_name:
            if on_sale:
                return round(price * 0.85, 2)
            else:
                return round(price, 2)
    return None
```

Question 14 [Difficulty: */**] [3 marks]

Write a function called `build_a_playlist` which has a parameter:

- `playlist` (type: `list[(str, str, str, int)]`): A list of tuples where each tuple contains the song title, the artist's name, the genre of the song and the release year.

  For example,
  ```
  playlist =
            [('Yellow', 'Coldplay', 'IndiePop', 2000),
             ('Photograph', 'Ed Sheeran', 'FolkPop', 2014),
             ('Cheer Up', 'Twice', 'K-Pop', 2016),
             ('Enchanted', 'Taylor Swift', 'CountryPop', 2010)]
  ```

This function will first prompt the user for each detail of a song (title, artist, genre and release year) and append it to `playlist`. Each song's details are to be stored as a tuple. The function will continue to prompt the user until the user enters an empty string (`''`) for the song title. You can assume that the year entered is always an integer.

The function returns the final playlist which is a list of tuples where each tuple contains the song's details (title, artist, genre and release year) as described above.

If the argument is an empty list and the user does not append any song to it, the function should return an empty list.

A sample run of the function can be found below:

```
Title: From the Start
Artist: Laufey
Genre: JazzPop
Release Year: 2023

Title: HOT
Artist: Seventeen
Genre: K-Pop
Release Year: 2022

Title: Espresso
Artist: Sabrina Carpenter
Genre: Funk
Release Year: 2024

Title:
```

Using the `playlist` variable and the sample run above:

- `build_a_playlist(playlist)` returns:

  ```
  [('Yellow', 'Coldplay', 'IndiePop', 2000), ('Photograph', 'Ed Sheeran',
  'FolkPop', 2014), ('Cheer Up', 'Twice', 'K-Pop', 2016), ('Enchanted',
  'Taylor Swift', 'CountryPop', 2010), ('From the Start', 'Laufey',
  'JazzPop', 2023), ('HOT', 'Seventeen', 'K-Pop', 2022), ('Espresso',
  'Sabrina Carpenter', 'Funk', 2024)]
  ```

Type your answer in the space provided on eLearn.

```
#Solution
def build_a_playlist (playlist):

    title = input("Title: ")

    while title != '':
        artist = input("Artist: ")
        genre = input("Genre: ")
        release_year = int(input("Release Year: "))

        song = (title, artist, genre, release_year)
        playlist.append(song)

        title = input("\nTitle: ")

    return playlist
```

Question 15 [6 marks]

Part A [Difficulty: **]

Write a function called `invert_actor_dictionary()` that accepts a mapping of actors to movies and returns a dictionary that maps movies to actors. It accepts:

- `actor_db` (type: `dict`) : a dictionary that maps actors (type: `str`) to a list of movie titles (type: `list[str]`).

The function should return:

- `movie_db` (type : `dict`) : a dictionary that maps movie titles (type: `str`) to a list of actors (`list[str]`). The order of actors in each list does not matter.

You may assume that the actors and movie titles are unique.

For example,

```
actor_db =
{'Christoph Waltz': ['Carnage 2011', 'Sieben Monde 1998'], 'Lance Henriksen': ['The
Nature of the Beast 1995', 'Aliens 1986', 'The Terminator 1984'], 'James Earl Jones':
['Robots 2005', 'Conan the Barbarian 1982', 'Clear and Present Danger 1994'], 'Michael
Biehn': ['They Wait 2007', 'The Seventh Sign 1988', 'The Terminator 1984'], 'Arnold
Schwarzenegger': ['Terminator: Dark Fate 2019', 'Terminator Genisys 2015', 'The
Terminator 1984', 'Conan the Barbarian 1982'], 'Sandahl Bergman': ['The Singing
Detective 2003', 'Conan the Barbarian 1982', 'All That Jazz 1979'], 'Linda Hamilton':
['The Stone Boy 1984', 'Missing in America 2005', 'The Terminator 1984']}
```

Example 1:

```
movie_db = invert_actor_dictionary(actor_db)
print(movie_db['The Terminator 1984'])
```

Prints:

```
['Linda Hamilton', 'Arnold Schwarzenegger', 'Michael Biehn', 'Lance Henriksen']
```

Example 2:

```
movie_db = invert_actor_dictionary(actor_db)
print(movie_db['Conan the Barbarian 1982'])
```

Prints:

```
['Arnold Schwarzenegger', 'James Earl Jones', 'Sandahl Bergman']
```

Write your answer in the space provided on eLearn.

```
# Solution
def invert_actor_dictionary(actor_db):
    dct = {}
    for actor, movies in actor_db.items():
        for movie in movies:
            if movie not in dct:
                dct[movie] = []
            dct[movie].append(actor)
    return dct
```

Part B [Difficulty: **/*]**

You are given a CSV file from IMDB containing the top 10,000 actors and the films in which they starred[1]. Write a function named `read_actor_db()`.

This function accepts one argument:

- `filename` (type: `str`) : The name of a file to read in CSV format. The first few lines of the file are in Table 1.
- Note that the file has a header line, which you must use to find the correct field in the CSV file. **IMBD may change the order of the columns, and your code should automatically adapt to this.**
- You may assume the field names will never change, but new fields may be added.
- You may assume that "," never appears in any field.
- You may assume that each line in the file is unique.

This function should return one value:

- `movie_db` (type: `dict`) : This dictionary must be structured to allow efficient retrieval of all the actors that star in a given movie and year.
    - The dictionary key is a tuple:
        - (`movie_title` (type: `str`), `year` (type: `int`))
    - The dictionary value lists the actors that starred in the movie.
        - `list [ actor` (type: `str`) `]`
        - The order of the list of actors does not matter.

```
Actor,ActorID,Film,Year,Votes,Rating,FilmID
Samuel L. Jackson,nm0000168,RoboCop,2014,217294,6.1,tt1234721
Arnold Schwarzenegger,nm0000216,Predator,1987,374744,7.8,tt0093773
Arnold Schwarzenegger,nm0000216,The Terminator,1984,803981,8,tt0088247
Arnold Schwarzenegger,nm0000216,Conan the Barbarian,1982,138445,6.9,tt0082198
Peter Weller,nm0000693,RoboCop,1987,235318,7.6,tt0093870
Kurtwood Smith,nm0001748,RoboCop,1987,235318,7.6,tt0093870
Joel Kinnaman,nm1172478,RoboCop,2014,217294,6.1,tt1234721 …
```

Table 1: Sample lines from IMDB 10K Actors Database.

For example, a CSV file named `samples.csv` containing only the lines in Table 1 is used as the input file:

```
movie_db = read_actor_db('samples.csv')
```

Example 1:

```
movie_title = 'RoboCop'
year = 1987
print(f'The actors in {movie_title} {year} are {movie_db[(movie_title, year)]}')
```

Prints:

```
The actors in RoboCop 1987 are ['Peter Weller', 'Kurtwood Smith']
```

Example 2:

```
movie_title = 'RoboCop'
year = 2014
print(f'The actors in {movie_title} {year} are {movie_db[(movie_title, year)]}')
```

Prints:

```
The actors in RoboCop 2014 are ['Samuel L. Jackson', 'Joel Kinnaman']
```

[1] IMDB Films by Actor for 10K Actors

Write your answer in the space provided on eLearn.

```
# Solution

def read_actor_db(filename):
    dct = {}
    line_num = 0
    with open(filename) as fp:
        # Read the database and populate the dictionary
        for line in fp:
            data = line.rstrip('\n').split(',')
            if line_num == 0:
                # Find the indices of the relevant fields
                for ix in range(len(data)):
                    if data[ix] == 'Film':
                        film_col = ix
                    if data[ix] == 'Actor':
                        actor_col = ix
                    if data[ix] == 'Year':
                        year_col = ix
            else:
                film_name = data[film_col]
                actor_name = data[actor_col]
                year = int(data[year_col])
                # Each film has a list of actors
                key = film_name, year
                if key not in dct:
                    dct[key] = []
                dct[key].append(actor_name)

            line_num = line_num + 1
    return dct
```

Question 16 [7 marks]

Part A [Difficulty: **]

Fill in no more than 4 lines of code below to get the following output:

```
[[], [4], [5, 4], [6, 5, 4], [7, 6, 5, 4], [8, 7, 6, 5, 4], [9, 8, 7, 6, 5, 4]]
```

```
res=[]
for i in range(3, 10):
    # add your code below




print(res)
```

Write your answer in the space provided on eLearn.

```
res=[]
for i in range(3, 10):

    # Solution 1 v1
    # res_i=[]
    # for j in range(i,3,-1):
    #     res_i.append(j)
    # res.append(res_i)

    # Solution 1 v2
    # res_i = [j for j in range(i, 3, -1)]
    # res.append(res_i)

    # Solution 1 v3
    # res.append([*range(i, 3, -1)])

    # Solution 2
    # if res!=[]:
    #     res.append([i]+res[-1])
    # else:
    #     res.append([])

    # Solution 3
    # res.append(list(range(4, i + 1))[::-1])

print(res)
```

Part B [Difficulty: **]

Trace the code step-by-step. Write down the final content of `output.txt` after executing `process_data(data)`, where `data = [ ["a", "3", 6, "b"], [4, "7", "c", "D", 10], ["e", 8, "5", "F"] ].`

```
def process_data(data):
    with open("output.txt", "w") as file:
        i = 0
        while i < len(data):
            result = ""
            inner_list = data[i]
            j = 0
            while j < len(inner_list):
                item = inner_list[j]
                if type(item) == str and item.isdigit():
                    result += str(int(item) * 2) + "-"
                elif type(item) == str and item.isalpha() and item.islower():
                    result += item.upper()
                elif type(item) == int and item % 2 == 0:
                    result += str(item // 2)
                j += 1
            i += 1
            file.write(result.rstrip("-") + "\n")

data = [ ["a", "3", 6, "b"], [4, "7", "c", "D", 10], ["e", 8, "5", "F"] ]
process_data(data)
```

Hint:

The `isdigit()` method returns `True` if all the characters of a string are digits, otherwise `False`.
The `isalpha()` method returns `True` if all the characters of a string are alphabet letters, otherwise `False`.
The `islower()` method returns `True` if all the alphabetic characters of a string are in lower case and there is at least one cased character in the string, otherwise `False`.

Write your answer in the space provided on eLearn.

```
A6-3B
214-C5
E410
```

Part C [Difficulty: **/*]**

Implement a function called `get_highest_level_courses` that takes in a parameter:

- `student_courses` (type: `dict`): This is a dictionary where keys are student names (type: `str`) and values are lists of course codes (type: `str`) that the student has completed. Each course code is a string in the format "`<course><level>`", where: `<course>` is a three-letter course identifier (e.g., "`MAT`" for Math, "`ENG`" for English, etc.). `<level>` is a two-digit number representing the level of the course (e.g., "`01`" for beginner, "`02`" for intermediate, "`03`" for advanced).

  E.g., `student_courses` may look like

```
{
     "Alice": ["MAT01", "ENG02", "MAT02", "SCI01"],
     "Bob": ["ENG01", "ENG02", "MAT01"],
     "Clara": ["SCI01", "SCI02", "SCI03", "ENG01"]
}.
```

The function should return a new dictionary where each key (type: `str`) is a student name, and the value is a list of the highest-level course code (type: `str`) for each unique course identifier they've completed. If `student_courses` is empty, then the function also returns an empty dictionary.

For example:

- If `student_courses` is

```
{"Alice": ["MAT01", "ENG02", "MAT02", "SCI01"],
 "Bob": ["ENG01", "ENG02", "MAT01"],
 "Clara": ["SCI01", "SCI02", "SCI03", "ENG01"]}
```

  then `get_highest_level_courses(student_courses)` returns

```
{'Alice': ['MAT02', 'ENG02', 'SCI01'],
 'Bob': ['ENG02', 'MAT01'],
 'Clara': ['SCI03', 'ENG01']}
```

  because for Alice, "`MAT02`" is her highest-level Math course, "`ENG02`" is her highest-level English course, and "`SCI01`" is her only Science course. For Bob, "`ENG02`" is his highest-level English course, and "`MAT01`" is his only Math course. For Clara, "`SCI03`" is her highest-level Science course, and "`ENG01`" is her only English course.

Note: A student may complete courses with same course code but different levels (e.g., "`ENG01`", "`ENG02`"). Your function **MUST NOT** exceed 10 lines of code, not including the `def` line. You cannot use imports.

Write your answer in the space provided on eLearn.

```
def get_highest_level_courses(student_courses):
    result = {}
    for student, courses in student_courses.items():
        highest = {}
        for course in courses:
            code, level = course[:3], course[3:]
            if code not in highest or int(level)> int(highest[code][3:]):
                highest[code] = course
        result[student] = list(highest.values())
    return result
```

Question 17 [Difficulty: ***] [4 marks]

`q17.py` on the next page is a buggy implementation of a question based on Q4 of Lab Test 2. Identify and correct ALL execution and logic errors (i.e., errors that cause the program to behave incorrectly when executed). One error has been identified for you.

Note: 1 mark will be deducted for every incorrect error that you have identified. The minimum score for this question is 0 mark.

The description of the question is as follows.

`ThinkCoin` is a popular cryptocurrency with a market value of over a trillion dollars.

As with physical currency, `ThinkCoin` coins have distinct values and are interchangeable.

For example, a user (represented by an address, see below) may have several coins of denominations `[5, 5, 1, 2]`. In a transaction that requires a `ThinkCoin` value of 9, the user must spend coins with value `[5, 5]` and receives change in the form of one coin with value 1. The user will now possess the following coins: `[1, 1, 2]`.

This problem models a sequence of `ThinkCoin` transactions.

Part A (Difficulty Level *)**
Implement a function called `get_invalid_transactions` in `q17.py`.

This function accepts one parameter:

- `filename` (`type: str`): A text file containing a set of `ThinkCoin` transactions. The format of the file is defined below.

This function returns:

- A list of invalid transaction IDs (type: `list[str]`) in the same order as in the text file. See below for the definitions of valid/invalid transactions. If there are no invalid transactions, the function returns an empty list.

Text File Format
The file contains a simplified representation of how funds move through a cryptocurrency system, from initial mining (coinbase) to various users via transactions. The transactions are ordered: the 1st line in the file represents the 1st transaction and so on. Each transaction is represented in the following format:

`TX_ID|IN:Address:Amount|…|OUT:Address:Amount|…`

Example transactions from a text file named `transactions_1.txt` are shown below.

```
TX001|IN:COINBASE:9|OUT:MinerA:9
TX002|IN:COINBASE:11|OUT:MinerB:11
TX003|IN:COINBASE:11|OUT:MinerB:11
TX004|IN:COINBASE:14|OUT:MinerC:14
TX005|IN:COINBASE:7|OUT:MinerD:7
TX006|IN:MinerA:9|OUT:MinerA:1|OUT:Alice:8
TX007|IN:MinerB:11|OUT:MinerB:1|OUT:Bob:10
TX008|IN:MinerA:9|OUT:Isaac:8|OUT:MinerA:1
```

```
TX009|IN:MinerC:14|IN:MinerD:7|OUT:Charlie:20|OUT:MinerC:1
TX010|IN:COINBASE:9|OUT:MinerA:9
TX011|IN:Alice:8|IN:Bob:10|OUT:David:12|OUT:Alice:1|OUT:Charlie:5
TX012|IN:Charlie:20|OUT:Eve:15|OUT:Charlie:5
TX013|IN:Charlie:2|OUT:Eve:2
TX014|IN:Eve:2|OUT:Jack:2
TX015|IN:MinerA:1|IN:MinerA:1|IN:Alice:1|OUT:Frank:1|OUT:David:2
```

- `TX_ID` represents the transaction ID, which is a unique string of letters and digits. The order of the transactions is determined by the line sequence in the file and not by the `TX_ID`.
- `IN` represents inputs (sources of funds). `OUT` represents outputs (recipients of funds) which only appear after all inputs. Each transaction has at least one input and one output.
- `Amount` represents the value of a `ThinkCoin` coin in transactions. It is always a positive integer.
- `Address` is a unique non-empty string which identifies a particular `ThinkCoin` user from whom coins are withdrawn (`IN`) or to whom coins are deposited (`OUT`).
- For each transaction, the total input amount is always equal to the total output amount.
- If the address is `COINBASE`, for example in `TX001` - the transaction is considered a coinbase transaction. This introduces new coins into circulation. Coinbase transactions have only one input which is `IN:COINBASE`, and only one output. Coinbase transactions are always valid.
- Coins received by an address (`OUT`) are distinct from other coins received at that address. For example, after `TX003`, `MinerB` has two different coins with value 11, each (received from `TX002` and `TX003`).
- Coins received by an address (`OUT`) that have equal value can be used interchangeably.
    - `Charlie` obtains two coins of value 5 from `TX011` and `TX012` respectively. These two coins are distinct but can be used interchangeably.
- Coins must be spent (`IN`) completely in a transaction. For example, in `TX007`, `MinerB` spends one coin with value 11, as `MinerB` has two coins of value 11, each. `MinerB` receives (`OUT`) a new coin with value 1 as change, and a coin of value 10 goes to `Bob`. After `TX007`, `MinerB` possesses two coins with values 1 and 11, respectively.
- All strings in the file are case-sensitive. There are no spaces between the characters in each line. The keywords `COINBASE`, `IN`, and `OUT` are not used for any other purposes.

All transactions are valid unless they violate this rule: an address **cannot** spend coins (`IN`) it does not possess, with the sole exception of `COINBASE`, which is able to create new coins. Invalid transactions are not processed. More examples below:

- After `TX001`, `MinerA` has one coin with value 9, which is then used in `TX006`. Both are valid transactions.
- Transaction `TX008` is **invalid** as `MinerA` has only 1 coin with value 1 after `TX006`. So `TX008` was not processed.
- `TX013` is **invalid** as `Charlie` possesses only two coins of value 5 each, obtained in previous, valid transactions `TX011` and `TX012`. `Charlie` cannot spend a coin of value 2 as `Charlie` does not have it.
- `TX014` is also **invalid** as `Eve` does not possess a coin of value 2 since `TX013` was not processed (invalid).
- `TX015` is **invalid** as `MinerA` possesses just one coin with value 1, and this transaction requires two coins with value 1, each.

`get_invalid_transactions('transactions_1.txt')` should therefore return:
`['TX008', 'TX013', 'TX014', 'TX015']`

The transaction IDs in the returned list are ordered according to the order in the text file.

```
    # q17.py

    # Line 000 contains an example of an error which has been identified
    # for you already on eLearn.
    # Start spotting errors from Line 001 onwards.


000 define get_invalid_transactions(filename):
001     '''
002     This function reads a text file containing a set of ThinkCoin
003     transactions and returns a list of invalid transactions.
004
005     Parameter:
006         filename (type: str): A text file containing a set of ThinkCoin
007         transactions.
008
009     Returns:
010         A list of invalid transaction IDs (type: list[str]) in the same
011         order as in the text file. See above for the definitions of
012         valid/invalid transactions. If there are no invalid transactions,
013         the function returns an empty list.
014     '''
015
016     invalid_transactions = []
017     miner_unspent_dict = {}
018
019     with open(filename, 'r') as my_file:
020         for line in my_file:
021             line = line.rstrip('\n')
022             line_list = line.split('|')
023             trans_id = line_list[0]
024
025             if 'COINBASE' in line:
026                 out = line_list[2]
027                 out_list = out.split('Miner')
028                 out_data_str = out_list[1]
029                 final_list = out_data_str.split(':')
030                 miner = 'Miner' + final_list[0]
031                 value = int(final_list[1])
032                 if miner not in miner_unspent_dict:
033                     miner_unspent_dict[miner] = {value:1}
034                 else:
035                     miner_value_dict = miner_unspent_dict[miner]
036                     if value not in miner_value_dict:
037                         miner_value_dict[value] = 1
038                     else:
039                         miner_value_dict[value] += 1
040             else:
041
042                 if not is_valid_transaction(miner_unspent_dict, line_list):
043                     invalid_transactions.append(trans_id)
044
045     return invalid_transactions
046
047 def is_valid_transaction(miner_unspent_dict, line_list):
```

```
048     # line_list is a list of strings capturing the IN and OUT transactions
049     # as built in the get_invalid_transactions() function.
050
051     tot_in = 0
052     tot_out = 0
053     transaction_in_dict = {}
054     transaction_out_dict = {}
055     for item in line_list:
056         item_list = item.split(':')
057         miner = item_list[1]
058         amount = item_list[2]
059         if 'IN' in item:
060             tot_in += amount
061             if miner not in transaction_in_dict:
062                 transaction_in_dict[miner] = 1
063             else:
064                 miner_amount_in_dict = transaction_in_dict[miner]
065                 if amount not in miner_amount_in_dict:
066                     miner_amount_in_dict[amount] = 1
067                 else:
068                     miner_amount_in_dict[amount] += 1
069
070         if 'OUT' in item:
071             tot_out += amount
072             if miner not in transaction_out_dict:
073                 transaction_out_dict[miner] = {amount:1}
074             else:
075                 miner_amount_out_dict = transaction_out_dict[miner]
076                 if amount not in miner_amount_out_dict:
077                     miner_amount_out_dict[amount] = 1
078                 else:
079                     miner_amount_out_dict[amount] += 1
080
081     # verification 1
082     if tot_in != tot_out:
083         return False
084
085     # verification 2
086     for miner in transaction_in_dict:
087         if miner not in miner_unspent_dict:
088             return False
089
090         miner_amount_in_dict = transaction_in_dict[miner]
091         miner_amount_unspent_dict = miner_unspent_dict[miner]
092
093         for amount in miner_amount_in_dict:
094             if amount not in miner_amount_unspent_dict:
095                 return False
096
097             num_coins_to_be_spent = miner_amount_in_dict[amount]
098             num_coins_that_can_be_spent = miner_amount_unspent_dict[amount]
099             if num_coins_to_be_spent > num_coins_that_can_be_spent:
100                 return False
101
102     # No rule is violated
103     update_valid_transaction(miner_unspent_dict, transaction_in_dict,
```

```
104          transaction_out_dict)
105      return True
106
107
108  def update_valid_transaction(miner_unspent_dict, transaction_in_dict,
109          transaction_out_dict):
110
111      for miner in transaction_in_dict:
112          miner_amount_in_dict = transaction_in_dict[miner]
113
114          for amount in miner_amount_in_dict:
115              num_coins = miner_amount_in_dict[amount]
116              miner_amount_unspent_dict = miner_unspent_dict[miner]
117              miner_amount_unspent_dict[amount] -= num_coins
118              if miner_amount_unspent_dict[amount] == 0:
119                  # the pop method removes the key from the dictionary
120                  # (and the related value)
121                  miner_amount_unspent_dict.pop(amount)
122
123      for miner in transaction_out_dict:
124          miner_amount_out_dict = transaction_out_dict[miner]
125
126          for amount in miner_amount_out_dict:
127              num_coins = miner_amount_out_dict[amount]
128              if miner not in miner_unspent_dict:
129                  miner_unspent_dict[miner] = [num_coins]
130              else:
131                  miner_amount_unspent_dict = miner_unspent_dict[miner]
132                  if amount not in miner_amount_unspent_dict:
133                      miner_amount_unspent_dict[amount] = num_coins
134                  else:
135                      miner_amount_unspent_dict[amount] += num_coins
136
137      final_miner_unspent_dict = {}
138      for key in miner_unspent_dict:
139          if miner_unspent_dict[key] != {}:
140              final_miner_unspent_dict[key] = miner_unspent_dict[key]
141
142      miner_unspent_dict = final_miner_unspent_dict
```

Write your answer in the space provided on eLearn.

```
# Solution
Error # 1:
Line 041, insert, line_list = line_list[1:]

Error #2:
Line 058 modify, amount = int(item_list[2])

Error # 3:
Line 062, modify, transaction_in_dict[miner] = {amount:1}

Error #4:
Line 129, modify, miner_unspent_dict[miner] = {amount:num_coins}
```

Question 18 [Difficulty: ***] [8 marks]

This question is a continuation of Q17.

Implement a function called `get_direct_transfers`.

This function accepts:

- A transaction (type: `str`) which is a `ThinkCoin` transaction as described in Q17, with several additional constraints defined below. The transaction is assumed to be always valid.

  Some example transactions are shown below. Note that they are not related to each other, i.e., each line represents a separate, valid transaction. The function will process a single transaction.

  ```
  TX001|IN:MinerC:7|IN:MinerD:8|OUT:Charlie:12|OUT:MinerD:2|OUT:MinerC:1

  TX002|IN:MinerA:10|OUT:Alice:8|OUT:MinerA:2

  TX003|IN:MinerA:14|OUT:MinerC:14

  TX004|IN:Alice:8|OUT:Bob:2|OUT:David:4|OUT:Jane:1|OUT:Alice:1

  TX005|IN:Cris:15|IN:Eve:14|OUT:Ann:10|OUT:Jane:12|OUT:Cris:4|OUT:Eve:3
  ```

  Additional constraints for each transaction, only applicable for Q18:

  - There is **at least one output address which is not an input address** in the transaction.
  - Any change returned to an address is **always smaller** than the input coin value for that address. For example, in `TX001`, `MinerC` spends one coin with value 7, and `MinerD` spends another coin with value 8. At the end of this transaction, `Charlie` receives (`OUT`) one coin with value 12, while `MinerC` and `MinerD` both receive a new coin with value 1 and 2 as changes, respectively.
  - There is **no repetition of addresses in the inputs, or the outputs**. Therefore inputs such as `IN:MinerA:2|IN:MinerA:3`, or outputs such as `OUT:MinerA:2|OUT:MinerA:4` will never be present in any transaction.

This function returns:

- A list (type: `list[(str, str, int)]`) of all direct transfers of coin values between pairs of different addresses in the transaction in the form of a tuple `(sender, receiver, amount)`. The `sender` must be different from the `receiver`. The `amount` is a positive integer. The order of tuples in the list must follow the order of addresses in the transaction. If no direct transfers can be determined from the transaction, the function returns an empty list. See examples below.

To identify direct transfers between different addresses, we only consider the following cases:

- There is only **one input and one output for the transaction, except for the change outputs**.
  - Consider `TX002`. `get_direct_transfers('TX002|IN:MinerA:10|OUT:Alice:8|OUT:MinerA:2')` returns `[('MinerA', 'Alice', 8)]` as the only direct transfer in this transaction, in which `Alice` receives a coin of value 8 from `MinerA` and `MinerA` receives a coin of value 2 back as change.

  - Consider `TX003`. There is no change output, so `get_direct_transfers('TX003|IN:MinerA:14|OUT:MinerC:14')` returns

`[('MinerA', 'MinerC', 14)]` as the only direct transfer in this transaction.

- There **are multiple inputs and only one output for the transaction, except for the change outputs**.
  - Consider `TX001`.
    `get_direct_transfers('TX001|IN:MinerC:7|IN:MinerD:8|OUT:Charlie:12|OUT:MinerD:2|OUT:MinerC:1')` returns `[('MinerC', 'Charlie', 6), ('MinerD', 'Charlie', 6)]` as two direct transfers in this transaction, in which `Charlie` receives two coins of value 6 from `MinerC` and `MinerD`, respectively. We can calculate these direct transfers based on the change outputs of value 2 and 1 for `MinerD` and `MinerC`, respectively. The order of tuples follows the order of addresses in the transaction, in which `MinerC` appears first.

- There is **only one input and multiple outputs for the transaction**.
  - Consider `TX004`.
    `get_direct_transfers('TX004|IN:Alice:8|OUT:Bob:2|OUT:David:4|OUT:Jane:1|OUT:Alice:1')` returns `[('Alice', 'Bob', 2), ('Alice', 'David', 4), ('Alice', 'Jane', 1)]` as the direct transfers in this transaction.

- The function returns an empty list for all other cases not listed above.
  - Consider `TX005`.
    `get_direct_transfers('TX005|IN:Cris:15|IN:Eve:14|OUT:Ann:10|OUT:Jane:12|OUT:Cris:4|OUT:Eve:3')` returns an empty list as there are more than one distinct input **and** more than one distinct output in the transaction, except for the change outputs.

Write your answer in the space provided on eLearn.

```
# Solution
def get_direct_transfers(transaction):
    direct_transfers = []
    # parse the valid transaction
    parts = transaction.split('|')
    inputs = [(p[3:].split(':')[0], int(p[3:].split(':')[1])) for p in parts if p.startswith('IN')]
    outputs = [(p[4:].split(':')[0], int(p[4:].split(':')[1])) for p in parts if p.startswith('OUT')]

    filtered_inputs = []
    change_outputs = []
    for input in inputs:
        in_address, in_amount = input
        has_change = False
        for output in outputs:
            out_address, out_amount = output
            if in_address == out_address:
                filtered_inputs.append((in_address, in_amount - out_amount))
                change_outputs.append(output)
                has_change = True
                break # only 1 output for each address
        if not has_change:
            filtered_inputs.append(input)
```

```
    filtered_outputs = []
    for output in outputs:
        if output not in change_outputs:
            filtered_outputs.append(output)
    # case 1: 1 input and 1 output
    if len(filtered_inputs) == 1 and len(filtered_outputs) == 1:
        direct_transfers.append((filtered_inputs[0][0], filtered_outputs[0]
[0], filtered_inputs[0][1]))

    # case 2: multiple inputs, 1 output
    if len(filtered_inputs) > 1 and len(filtered_outputs) == 1:
        for input in filtered_inputs:
            direct_transfers.append((input[0], filtered_outputs[0][0], input
[1]))

    # case 3: multiple outputs, 1 input
    if len(filtered_inputs) == 1 and len(filtered_outputs) > 1:
        for output in filtered_outputs:
            direct_transfers.append((filtered_inputs[0][0], output[0], output
[1]))

    # other: return []
    return direct_transfers
```

END OF PAPER

THIS PAGE IS INTENTIONALLY LEFT BLANK.

THIS PAGE IS INTENTIONALLY LEFT BLANK.

THIS PAGE IS INTENTIONALLY LEFT BLANK.